

ONEDRAFT

CAD-AI — Aria Powered

MVP Technical Stack & Repository Architecture

Version 0.1

Status: Engineering Foundation

Product: OneDraft

AI Layer: Aria Powered

Architecture: Model-Centric / Service-Oriented / Versioned

Primary Objective: Build the first production-capable specification-to-coordinated-drawing loop

1. ENGINEERING OBJECTIVE

The OneDraft MVP shall establish a complete working production loop:

CUSTOMER SPECIFICATION

↓

ARIA INTERPRETATION

↓

STRUCTURED PROJECT MODEL

↓

PARAMETRIC GEOMETRY

↓

ARCHITECTURAL VIEWS

↓

ARCH D DOCUMENTATION

↓

VALIDATION

↓

CUSTOMER REDLINE

↓

MODEL REVISION

↓

DRAWING REGENERATION

↓

CUSTOMER ACCEPTANCE

↓

FINAL DOCUMENT PACKAGE

The MVP must prove this loop before expanding into the full enterprise architecture.

2. RECOMMENDED TECHNOLOGY STRATEGY

The initial implementation should use technologies with strong support for:

Python

TypeScript

PostgreSQL

WebGL

3D geometry

PDF generation

containerized deployment

asynchronous workers

AI APIs

The objective is not to select the largest possible technology stack.

The objective is to minimize unnecessary complexity while preserving the architectural boundaries already established.

3. PRIMARY APPLICATION STACK

The initial stack should consist of:

Frontend

Next.js / React

Used for:

Project dashboard
Specification interface
AI workspace
Drawing viewer
Model viewer
Redline interface
Project administration

Backend API

Python + FastAPI

Used for:

Project APIs
Model operations
AI orchestration
Validation
Drawing generation orchestration
Revision management
Authentication integration

Python is particularly suitable because the geometry, AI, document-processing and computational ecosystem is substantial.

Database

PostgreSQL

Used as the principal transactional database.

Object Storage

S3-compatible object storage

Used for:

PDF
CAD exports
Images
Uploaded references
Generated artifacts
Archives

Queue

Redis-backed job queue initially

with the architecture designed so that it can later transition to a larger message-broker system if required.

Containers

Docker

All backend services and workers should be containerized.

4. GEOMETRY STRATEGY

This is one of the most important engineering decisions.

OneDraft should not initially attempt to build an entirely new CAD kernel.

Instead, the MVP should maintain its own **semantic architectural model** and use established computational geometry technology underneath it.

The conceptual stack is:

OneDraft Semantic Model

↓

Geometry Abstraction Layer

↓

Computational Geometry Kernel

↓

2D / 3D Representation

The abstraction layer prevents the entire OneDraft system from becoming permanently dependent on one geometry implementation.

5. GEOMETRY KERNEL

The initial geometry implementation should investigate an open computational geometry kernel such as:

Open CASCADE Technology

or another equivalent robust solid-modeling kernel appropriate to the final licensing requirements.

The geometry abstraction layer should expose operations such as:

```
create_wall()  
create_floor()  
create_roof()  
create_opening()  
create_stair()  
create_column()  
intersect()  
subtract()  
union()  
section()  
project()  
measure()
```

The rest of OneDraft should not need to know the internal kernel implementation.

6. SEMANTIC ARCHITECTURAL MODEL

The MVP model should define its own domain objects.

Examples:

Project

Site

Building

Level

Grid

Space

Wall

Door

Window

Floor

Roof

Stair

Ramp

Column

Opening

Assembly

Constraint

Dimension

View

Drawing

Sheet

Revision

Redline

This is the heart of the system.

7. DATABASE STRATEGY

PostgreSQL becomes the authoritative transactional store.

The initial schema should be organized around major domains rather than one enormous table.

Core schemas:

identity

projects

model

geometry

documentation

compliance

revisions

audit

billing

8. IDENTITY DOMAIN

Primary entities:

users

organizations

organization_members

roles

permissions

Example:

Organization

↓

Member

↓

Project

The organization boundary becomes the fundamental multi-tenant security boundary.

9. PROJECT DOMAIN

Core tables:

projects

project_sites

project_types

project_settings

project_requirements

project_status

The project record contains the immutable Project_ID and references the current model version.

10. MODEL DOMAIN

Core tables:

buildings
levels
grids
spaces
elements
assemblies
constraints
relationships
parameters

Each object receives a persistent UUID.

Example:

WALL - 00427

must continue to represent the same semantic wall through successive model revisions unless that element is explicitly destroyed and recreated.

11. ELEMENT TABLE

Conceptually:

elements

contains:

id

project_id

element_type

level_id

parent_id

geometry_reference

assembly_id

parameter_set

status

created_revision

modified_revision

deleted_revision

The geometry itself should not necessarily be embedded directly into the primary relational record.

12. GEOMETRY STORAGE

Geometry should be represented through a geometry abstraction layer.

Possible storage approaches include:

serialized geometry

binary geometry references

mesh references

kernel-native representations

2D drawing representations

The relational database stores the identity and relationship.

Object storage can store large generated geometry artifacts where appropriate.

13. REQUIREMENT MODEL

The MVP should implement:

requirements

with:

id

project_id

source

description

type

priority

mandatory

status

verification_state

created_revision

modified_revision

This permits Aria to distinguish customer requirements from its own generated design decisions.

14. CONSTRAINT MODEL

constraints

contains:

id

project_id

constraint_type

source

value

unit

tolerance

priority

affected_elements

status

This becomes the foundation for parametric behavior.

15. RELATIONSHIP GRAPH

The system shall maintain relationships independently from raw geometry.

Examples:

WALL - 001

bounds

ROOM - 004

DOOR - 003

hosted_by

WALL - 001

WINDOW-004

located_on

WALL-001

SECTION-A401

references

LEVEL-L1

This relationship graph is essential for intelligent regeneration.

16. REVISION DATABASE

Core tables:

model_revisions

revision_changes

revision_snapshots

redlines

redline_resolutions

A revision records what changed rather than simply overwriting the previous state.

17. EVENT STORE

The MVP should maintain an append-only project event table.

Example:

project_events

Fields:

event_id

project_id

event_type

actor_id

revision_id

payload

timestamp

Example event:

ELEMENT_MODIFIED

This creates the beginning of the OneDraft provenance system.

18. ARIA INTEGRATION

Aria shall not receive unrestricted access to PostgreSQL.

Instead:

Aria

↓

Command Generator

↓

Command Validator

↓

Project Model API

↓

Database Transaction

The AI therefore operates through a controlled interface.

19. ARIA COMMAND SCHEMA

A command should conceptually resemble:

command_id

project_id

revision_id

operation

target_objects

parameters

constraints_to_preserve

reason

requested_by

confidence

validation_requirements

The command is evaluated before execution.

20. COMMAND EXAMPLE

Customer instruction:

Move the east wall 600 mm east while maintaining the building footprint.

Aria converts this to an operation equivalent to:

MOVE_ELEMENT

Target:

WALL - 00427

Displacement:

+600 mm X

Preserve:

EXTERIOR_ENVELOPE

Then the command engine evaluates dependencies.

If the command would violate an immutable constraint, the transaction is not silently committed.

21. AI RESPONSE TYPES

Aria should be able to return more than prose.

The internal response categories should include:

ANSWER

COMMAND

CLARIFICATION

WARNING

VALIDATION_RESULT

DESIGN_PROPOSAL

ERROR

REVIEW_REQUIRED

This allows the UI to respond appropriately.

22. MODEL TRANSACTION

A model modification should follow:

BEGIN TRANSACTION

↓

Load current project state

↓

Validate requested command

↓

Apply change

↓

Update dependencies

↓

Regenerate affected geometry

↓

Run required validation

↓

Run drawing impact analysis

↓

Commit or reject

↓

Create revision

↓

Create audit event

This is the core transactional behavior of OneDraft.

23. DEPENDENCY ENGINE

The dependency engine determines what must change after a model modification.

Example:

Wall moved

↓

Room boundary changed

↓

Room area changed

↓

Door position changed

↓

Dimension changed

↓

Section intersection changed

↓

Elevation changed

↓

Drawing affected

Only affected outputs should be regenerated where possible.

24. DRAWING GENERATION SERVICE

The Drawing Service should expose operations such as:

`generate_plan()`

`generate_elevation()`

`generate_section()`

`generate_detail()`

`generate_schedule()`

`compose_sheet()`

`generate_document_set()`

The service consumes model state.

It does not become another source of truth.

25. ARCH D DOCUMENT ENGINE

The MVP shall include an ARCH D document template system.

A sheet template contains:

Paper Size

Title Block

Project Information

Sheet Number

Sheet Title

Revision Block

Graphic Standards

Drawing Regions

The actual drawing content is populated from model-generated views.

26. PDF PIPELINE

The PDF pipeline should be:

MODEL

↓

VIEW

↓

VECTOR DRAWING

↓

SHEET COMPOSITION

↓

PDF

↓

PDF QA

↓

HASH

↓

OBJECT STORAGE

The final PDF receives a content hash for provenance.

27. CAD EXPORT

CAD interoperability should be treated as a separate service.

Initial target:

DXF

followed by additional CAD/BIM formats after the semantic model has matured.

The export layer converts OneDraft objects into external representations.

The internal Project Model remains authoritative.

28. MODEL VIEWER

The web application should include a lightweight model viewer.

The viewer displays:

3D Building

Levels

Spaces

Elements

Sections

Selections

Measurements

Model Changes

The viewer does not need to become a complete desktop CAD application in the MVP.

It needs to provide enough interaction for customers to inspect and redline the generated model.

29. DRAWING VIEWER

The drawing viewer should support:

Zoom

Pan

Sheet Navigation

Layer Visibility

Measurement

Markup

Redline

Revision Comparison

Search

The viewer should allow a customer to select an element in a drawing and identify the underlying model object where feasible.

30. REDLINE IMPLEMENTATION

A redline consists of:

Visual Markup

plus:

Structured Instruction

The visual markup is therefore only the interface.

The structured redline becomes the actual model instruction.

Example:

Customer draws:

→

and writes:

Move this wall 600 mm.

The system stores:

REDLINE - 0091

Target:

WALL - 00427

Instruction:

MOVE +600mm

Status:

OPEN

31. CODE SERVICE — MVP

The first implementation should not attempt to encode every building code in existence.

Instead, the service should establish the architecture for jurisdiction-specific rule sets.

The workflow is:

Project Address

↓

Jurisdiction Resolver

↓

Applicable Regulatory Sources

↓

Source Version

↓

Code Manifest

↓

Rule Dataset

↓

Validation

The architecture must support adding jurisdictions later without redesigning the core project model.

32. CODE MANIFEST TABLE

`code_manifests`

Fields:

id

project_id

jurisdiction

sources

versions

effective_dates

retrieval_timestamp

content_hash

status

The manifest becomes associated with each applicable project revision.

33. VALIDATION SERVICE

The Validation Service exposes:

`validate_project()`

`validate_element()`

`validate_requirement()`

`validate_drawing_set()`

`validate_revision()`

Results are stored rather than returned only as temporary AI text.

34. VALIDATION RESULT

Each result contains:

rule_id

project_id

affected_objects

status

severity

description

source

revision

timestamp

Possible statuses:

PASS

FAIL

WARNING

REVIEW_REQUIRED

NOT_APPLICABLE

35. JOB SYSTEM

The MVP shall use an asynchronous job system.

Job examples:

MODEL_GENERATION

GEOMETRY_REBUILD

CODE_VALIDATION

DRAWING_GENERATION

PDF_GENERATION

CAD_EXPORT

RENDER

DOCUMENT_QA

Each job has:

Job_ID

Project_ID

Type

Priority

Status

Progress

Created

Started

Completed

Error

36. WORKER ARCHITECTURE

Workers should be independently deployable.

Initial workers:

AI Worker

Geometry Worker

Validation Worker

Drawing Worker

Document Worker

Render Worker

This allows computationally expensive operations to scale independently.

37. REPOSITORY STRUCTURE

The initial monorepo should be organized conceptually as:

onedraft/

apps/

web/

api/

services/

aria/

project_model/

geometry/
compliance/
validation/
documentation/
rendering/
exports/
workers/
database/
packages/
schemas/
ui/
config/
tests/
docs/
infra/

38. FRONTEND

apps/web

Contains:

Dashboard

Project Workspace

Specification Interface

AI Chat

Model Viewer

Drawing Viewer

Redline Interface

Code Panel

Revision Viewer

Account

The frontend should communicate only through defined APIs.

39. API

apps/api

Contains:

Authentication

Project endpoints

Model endpoints

Command endpoints

Drawing endpoints

Redline endpoints

Revision endpoints

Validation endpoints

Export endpoints

Billing endpoints

The API becomes the controlled gateway into project operations.

40. ARIA SERVICE

services/aria

Contains:

Prompt/Instruction Processing

Context Assembly

Tool Definitions

Command Generation

Reasoning Orchestration

Response Parsing

Confidence / Review Logic

The service should be model-provider agnostic.

This is important.

OneDraft should not make its entire architecture dependent upon one underlying AI provider.

41. PROJECT MODEL SERVICE

services/project_model

Contains:

Domain Objects

Model Commands

Transactions

Relationship Management

Versioning

Snapshots

Dependency Resolution

This is the most important backend domain.

42. GEOMETRY SERVICE

services/geometry

Contains:

Geometry Kernel Adapter

Architectural Primitives

Boolean Operations

Sectioning

Projection

Measurement

Mesh Generation

Geometry Serialization

The kernel should be hidden behind the adapter.

43. COMPLIANCE SERVICE

services/compliance

Contains:

Jurisdiction Resolver

Source Registry

Code Document Registry

Rule Extraction

Rule Applicability

Code Manifest

Regulatory Versioning

44. DOCUMENTATION SERVICE

services/documentation

Contains:

View Generation

Drawing Generation

Sheet Templates

Annotations

Dimensions

Schedules

Title Blocks

Document Composition

45. RENDERING SERVICE

services/rendering

Contains:

3D Scene Construction

Camera

Materials

Lighting

Technical Render

Architectural Visualization

The rendering service remains independent from the CAD documentation engine.

46. EXPORT SERVICE

services/exports

Contains:

PDF

DXF

Future:

DWG

IFC

BIM Interoperability

The export service translates the OneDraft model into external formats.

47. DATABASE MIGRATIONS

Database schema changes shall be version controlled.

Example:

migration_001_initial

migration_002_requirements

migration_003_model_elements

migration_004_revisions

This ensures deployments remain reproducible.

48. TESTING ARCHITECTURE

Testing must occur at several levels.

Unit Tests

Test individual domain functions.

Geometry Tests

Verify geometry operations.

Model Tests

Verify relationships and constraints.

AI Command Tests

Verify that natural-language requests become valid structured commands.

Integration Tests

Verify services working together.

Drawing Tests

Verify generated sheets.

Regression Tests

Verify that previously working projects continue to generate correctly.

49. GOLDEN PROJECTS

OneDraft should maintain a collection of canonical test projects.

Examples:

Single-Storey Residence

Two-Storey Residence

Basement Residence

Multi-Unit Building

Commercial Building

Complex Multi-Level Structure

The stadium project can ultimately become one of the high-complexity regression projects.

Every significant engine modification should regenerate the golden projects and compare results.

50. AI EVALUATION

AI quality cannot be evaluated solely by asking:

“Does the answer sound good?”

Evaluation should include:

Command Validity

Model Consistency

Constraint Preservation

Requirement Satisfaction

Geometry Correctness

Drawing Coordination

Revision Accuracy

Regulatory Reasoning

Hallucination Detection

AI-generated model commands must therefore be testable independently of conversational prose.

51. DEVELOPMENT ENVIRONMENT

Developers should be able to run the MVP locally using containerized dependencies.

Conceptually:

Web

API

PostgreSQL

Redis

Object Storage

Workers

should be launchable as one development environment.

This reduces onboarding friction and keeps local development close to production architecture.

52. DEPLOYMENT TOPOLOGY

The initial deployment should resemble:

Internet

↓

CDN / HTTPS

↓

Web Application

↓

API Gateway

↓

Application Services

↓

Job Queue

☑ ↓ ☑

AI | Geometry | Documentation Workers

↓

PostgreSQL

Object Storage

Audit/Event Store

This architecture can later be distributed across multiple availability zones and regions.

53. OBSERVABILITY

The system should implement:

Application Logs

Structured Error Logs

Metrics

Job Metrics

API Latency

AI Usage

Geometry Runtime

Drawing Generation Time

Validation Failures

Storage Usage

System Health

Every computational job should be traceable back to a Project_ID.

54. SECURITY BOUNDARY

The most important security boundaries are:

User

↓

Organization

↓

Project

↓

Model

↓

Document

Every access request must be authorized against those relationships.

A user must never gain access to another organization's project merely because they know an object identifier.

55. DATA OWNERSHIP MODEL

OneDraft should distinguish:

Customer Project Data

OneDraft Software

AI Model / Service

Generated Artifacts

System Metadata

The contractual treatment of each category should be established separately rather than treating everything as one undifferentiated asset.

56. FINAL ACCEPTANCE PIPELINE

The final pipeline becomes:

MODEL

↓

VALIDATION

↓

DRAWING QA

↓

DOCUMENT PACKAGE

↓

DOCUMENT HASHING

↓

CUSTOMER REVIEW

↓

REDLINE

↓

REVISION

↓

REVALIDATION

↓

FINAL CUSTOMER ACCEPTANCE

↓

ARCHIVE

This produces a complete traceable lifecycle.

57. MVP REPOSITORY — MASTER STRUCTURE

The initial implementation therefore resolves conceptually to:

onedraft/

apps/

- web
- api

services/

- aria
- project_model
- geometry
- compliance
- validation
- documentation
- rendering
- exports

workers/

- ai
- geometry
- validation
- drawing
- document
- render

database/

- migrations
- seed
- schemas

packages/

- domain
- schemas
- ui
- geometry_types
- api_client

tests/

- unit
- integration
- geometry
- ai
- drawing
- regression
- golden_projects

infra/

- docker
- deployment
- monitoring
- security

docs/

- architecture
- api
- model
- compliance
- operations

58. FIRST BUILD TARGET

The first actual development milestone should not be the complete architectural drawing package.

It should be:

PROJECT CREATION → MODEL → SIMPLE BUILDING → DRAWING

A user creates:

2-storey residence

The system creates:

Site

Ground Level

Upper Level

Walls

Doors

Windows

Rooms

Floor slabs

Roof

The system then generates:

Ground Floor Plan

Upper Floor Plan

Four Elevations

One Section

and produces:

ARCH D PDF

If that works from a structured project model, the foundation is proven.

59. SECOND BUILD TARGET

Then implement:

Natural-language modification

Example:

“Move the rear bedroom wall 300 mm north and enlarge the bedroom while keeping the exterior envelope unchanged.”

Aria generates a model command.

The model changes.

Dependencies update.

Affected drawings regenerate.

The customer sees the revision.

That proves that CAD-AI is actually model-aware.

60. THIRD BUILD TARGET

Then implement:

Redline

Customer marks:

“Add another window here.”

Aria interprets the redline.

The system creates the window.

The elevation updates.

The floor plan updates.

The window schedule updates.

The section updates if affected.

This demonstrates the defining OneDraft capability:

one model → coordinated documentation.

61. FOURTH BUILD TARGET

Introduce the first jurisdictional Code Manifest workflow.

The system determines:

Where is the project?

Which regulatory framework applies?

Which versions are applicable?

What rules affect this project?

The system records the resulting manifest before design generation.

The compliance engine then evaluates the project against the configured rules.

62. FIFTH BUILD TARGET

Introduce:

Customer acceptance

The customer receives the complete document package.

The system records:

Project Revision

Model Version

Drawing Set

Validation State

Code Manifest

Redline State

Document Hashes

Acceptance Event

The MVP now has a complete commercial transaction lifecycle.

63. ENGINEERING PRINCIPLE

The implementation should always preserve the following hierarchy:

CUSTOMER INTENT

↓

ARIA INTERPRETATION

↓

STRUCTURED COMMAND

↓

PROJECT MODEL

↓

GEOMETRY

↓

VALIDATION

↓

DOCUMENTATION

↓

CUSTOMER REVIEW

↓

REVISION

↓

ACCEPTANCE

No shortcut should bypass the Project Model.

That is the architectural rule that protects OneDraft from becoming an AI image-generation application with CAD-like decoration.

64. VERSION 0.1 DECISION

The OneDraft MVP technical foundation is therefore established as:

Frontend: React / Next.js

Backend: Python / FastAPI

Primary Database: PostgreSQL

Object Storage: S3-compatible

Queue: Redis-backed asynchronous jobs

Deployment: Docker/containerized

AI: Provider-agnostic Aria integration layer

Geometry: Semantic OneDraft model + geometry abstraction + computational geometry kernel

Documentation: Model-derived vector drawing pipeline

Output: ARCH D PDF initially, followed by CAD/BIM interoperability

Architecture: Modular monorepo initially, independently scalable services/workers

Persistence: Versioned Project Model + event/audit history

Core Principle: Project Model remains the single source of truth.

65. ENGINEERING DEFINITION

At this point OneDraft has progressed through four architectural levels:

LEVEL 1 — PRODUCT CONCEPT

CAD-AI — Aria Powered.

LEVEL 2 — PROJECT DATA MODEL

The building becomes a persistent structured computational model.

LEVEL 3 — SYSTEM ARCHITECTURE

AI, model, geometry, compliance, validation and documentation become independent coordinated subsystems.

LEVEL 4 — MVP ENGINEERING ARCHITECTURE

The subsystems now have defined technology boundaries, repositories, APIs, persistence, workers and deployment topology.

The next stage is therefore no longer conceptual.

It is the definition of the **actual OneDraft domain schema and API contract**.

That stage will specify the concrete objects and operations that the software developers—or a future automated coding system—will implement.

Version 0.1 — MVP Technical Stack & Repository Architecture established.